



Kohou Bi Irié Franck Aymar Romaric

Formateur IA, ML & IoT

CHAPITRE COMPLET

Statistiques Descriptives, Inférentielles & Séries Temporelles

avec NumPy · Pandas · SciPy

Niveau : Débutant → Avancé · 25 sections · Exemples réels · Code Python prêt à l'emploi

Ce cours constitue une base solide pour comprendre les statistiques utilisées quotidiennement en **Data Science**, **Machine Learning** et **Intelligence Artificielle**. Chaque concept est expliqué en profondeur avec sa formule, son interprétation, un exemple concret africain/professionnel, et le code Python correspondant.

01	Introduction à la Statistique
	• Rôle, domaines d'application • Questions statistiques réelles
02	Types de Données
	• 2.1 Qualitatives : Nominale & Ordinale • 2.2 Quantitatives : Discrète & Continue • Tableau données → outils adaptés
03	Échelles de Mesure
	• 3.1 Nominale • 3.2 Ordinale • 3.3 Intervalle • 3.4 Ratio • Tableau de synthèse des 4 échelles
04	Population et Échantillon
	• 4.1 Aléatoire simple • 4.2 Systématique • 4.3 Stratifié • 4.4 Par grappes • 4.5 Convenance • Tableau récapitulatif des méthodes
05	Types de Statistiques
	• Descriptive vs Inférentielle
06	Mesures de Tendance Centrale
	• 6.1 Moyenne • 6.2 Médiane • 6.3 Mode
07	Mesures de Dispersion
	• 7.1 Écart moyen • 7.2 Variance • 7.3 Écart-type • Impact des outliers sur la dispersion
08	Loi Normale & Théorème Central Limite
	• 8.1 Loi normale — règle 68-95-99.7% • 8.2 TCL — pourquoi les tests paramétriques fonctionnent
09	Erreur Standard
	• Formule $SE = s / \sqrt{n}$ • Différence écart-type vs erreur standard
10	Asymétrie & Kurtosis
	• 10.1 Skewness — asymétrie • 10.2 Kurtosis — épaisseur des queues
11	Test de Normalité : Shapiro-Wilk
	• H_0 / H_1 • Shapiro-Wilk & D'Agostino-Pearson • Arbre de décision paramétrique vs non paramétrique
12	Covariance
	• Formule $Cov(X, Y)$ • Lien covariance → corrélation • Matrice de covariance numpy & pandas
13	Corrélation

- 13.1 Pearson — relation linéaire
- 13.2 Spearman — relation monotone
- Interprétation des coefficients r

14 Visualisation Statistique

- Histogramme, Boxplot, Barres, Scatter
- Heatmap de corrélation, QQ-plot
- Dashboard complet avec matplotlib

15 Tests d'Hypothèse

- H_0 et H_1
- Logique du test statistique

16 p-value & Intervalle de Confiance

- Table de décision p-value
- IC 90% / 95% / 99% avec scipy

17 Test t de Student

- 1 échantillon
- 2 échantillons indépendants
- Test apparié

18 ANOVA — Analyse de la Variance

- F-score
- Post-hoc Tukey HSD
- Pourquoi pas plusieurs tests t ?

19 Test du χ^2 (χ^2)

- Tableau de contingence
- `chi2_contingency`

20 Tests Non Paramétriques

- 20.1 Mann-Whitney U — alternative au test t
- 20.2 Kruskal-Wallis — alternative à l'ANOVA

21 Z-score, IQR et Détection d'Outliers

- 21.1 Z-score — formule et seuils
- 21.2 IQR — Clôtures de Tukey
- VS Z-score vs IQR

22 Chargement des Données avec Pandas

- `read_csv` / `read_excel` / `read_json`
- `shape`, `info`, `head`, `describe`, `dtypes`

23 Nettoyage des Données

- Doublons, types, NaN
- Valeurs incohérentes

24 Arbre de Décision des Tests Statistiques

- Tableau situation → test recommandé
- Les 3 questions à se poser

25 Pipeline Complet d'Analyse de Données

- Les 7 étapes : Collecter → Conclure
- Code pipeline de A à Z
- Récapitulatif final : situation → outil

SECTION 01

Introduction à la Statistique

La **statistique** est la science qui permet de **collecter, organiser, résumer, analyser** et **interpréter** des données afin de prendre de meilleures décisions.

La statistique est le socle de :

Domaine	Application statistique
Data Science	Exploration et modélisation de données massives
Intelligence Artificielle	Entraînement, évaluation et interprétation des modèles
Économie & Finance	Prévisions, risques, indicateurs macroéconomiques
Recherche scientifique	Validation d'hypothèses, essais cliniques
Machine Learning	Standardisation, sélection de variables, détection d'anomalies

Exemples réels de questions statistiques

- *Quel est le salaire moyen des employés de cette entreprise ?*
 - *Les ventes augmentent-elles d'un mois à l'autre ?*
 - *Une campagne publicitaire a-t-elle réellement amélioré les ventes ?*
 - *Quel sera le chiffre d'affaires du mois prochain ?*
 - *Le prix du cacao à Abidjan suit-il une tendance saisonnière ?*
- Toutes ces questions relèvent de la statistique.

SECTION 02

Types de Données

Identifier le **type de données** est la première étape de toute analyse. Il détermine quelles statistiques calculer, quels graphiques tracer et quels tests statistiques appliquer.

Arbre de classification des données

DONNÉES			
Qualitatives (Catégorielles)		Quantitatives (Numériques)	
Nominale	Ordinale	Discrète	Continue
Sans ordre Genre, Dept, Couleur	Avec ordre Satisfaction, Niveau études	Valeurs entières nb enfants, nb transactions	Valeurs réelles salaire, taille, température

3c.1

Données Qualitatives (Catégorielles)

Les données qualitatives décrivent des **caractéristiques non numériques**. Elles représentent des catégories ou des classes.

Nominale	Ordinale
Catégories sans ordre logique . Aucune valeur n'est supérieure à une autre.	Catégories avec un ordre logique , mais sans distance uniforme entre les rangs.
Exemples : Genre, Nationalité, Département, Type de contrat, Couleur, Marque de téléphone.	Exemples : Satisfaction (1→5), Niveau d'études, Taille de T-shirt (XS→XL), Classement.
Mesures : Mode, Fréquence, Chi², Graphique en barres.	Mesures : Médiane, Mode, Spearman, Mann-Whitney.

3c.2

Données Quantitatives (Numériques)

Les données quantitatives sont des **valeurs numériques mesurables**. On peut effectuer des opérations arithmétiques dessus.

Discrète	Continue
Prend des valeurs entières dénombrables. Il ne peut pas exister de valeur intermédiaire.	Peut prendre toute valeur réelle dans un intervalle. La précision dépend de l'instrument de mesure.
Exemples : Nombre d'enfants (0, 1, 2, 3...), Nombre de transactions, Nombre de défauts, Âge en années entières.	Exemples : Salaire (350 250,75 FCFA), Taille (1,732 m), Température (37,4°C), Poids, Durée exacte.
Mesures : Moyenne, Mode, Variance, Histogramme (classes entières).	Mesures : Toutes — Moyenne, Médiane, Variance, Écart-type, Histogramme, Densité de probabilité.

Tableau récapitulatif — Type de données → Outils adaptés

Type	Sous-type	Exemples concrets	Statistiques	Graphique adapté
Qualitative	Nominale	Genre, Département, Pays	Mode, Chi ²	Barres, Camembert
Qualitative	Ordinale	Satisfaction, Niveau, Classement	Médiane, Spearman	Barres ordonnées
Quantitative	Discrète	Nb enfants, Nb transactions	Moyenne, Variance	Histogramme entier
Quantitative	Continue	Salaire, Taille, Température	Tous les tests paramétriques	Histogramme, Boxplot

■ pandas — identifier les types de données automatiquement

```
import pandas as pd
import numpy as np
df = pd.read_csv('employees.csv')
# Types pandas bruts
print(df.dtypes)
# Séparation automatique par type
quanti = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
quali = df.select_dtypes(include=['object', 'category']).columns.tolist()
print('Quantitatives :', quanti)
print('Qualitatives :', quali)
# Résumé selon le type
print(' --- Stats quantitatives ---')
print(df[quanti].describe().round(2))
print(' --- Stats qualitatives ---')
for col in quali:
    print(f'{col} : {df[col].nunique()} modalités | Mode = {df[col].mode()[0]}')
    print(df[col].value_counts(normalize=True).round(2))
    print()
```

SECTION 03

Échelles de Mesure

L'**échelle de mesure** d'une variable détermine quelles opérations mathématiques sont autorisées et quels tests statistiques peuvent être appliqués. Utiliser le mauvais test sur une mauvaise échelle produit des résultats sans sens.

3b.1

Nominale — catégories sans ordre

Les données sont des **catégories sans ordre ni distance**. On peut uniquement compter les occurrences. Aucune opération arithmétique n'est possible.

Exemples & règle

• Genre : Homme / Femme • Département : IT / RH / Finance

• Couleur préférée, nationalité, type de contrat

→ Calculer la moyenne du genre n'a aucun sens. Seule la fréquence est pertinente.

Opérations autorisées : comptage, mode, fréquence relative, χ^2 .

■ pandas — nominale

```
import pandas as pd
df = pd.DataFrame({'dept': ['IT', 'RH', 'IT', 'Finance', 'IT', 'RH']})
print(df['dept'].value_counts()) # fréquences
print(df['dept'].value_counts(normalize=True).round(2)) # proportions
# Encodage One-Hot pour ML
print(pd.get_dummies(df['dept'], prefix='dept'))
```

3b.2

Ordinale — ordre sans distance uniforme

Les catégories ont un **ordre logique**, mais la **distance entre rangs n'est pas uniforme**. On peut trier et comparer, mais pas calculer une moyenne significative.

Exemples & règle

• Satisfaction : Très insatisfait < Insatisfait < Neutre < Satisfait < Très satisfait

• Niveau d'études : Primaire < Secondaire < Licence < Master < Doctorat

→ La différence entre Satisfait (4) et Insatisfait (2) n'est pas 2 unités réelles.

Opérations autorisées : médiane, percentiles, Spearman, Mann-Whitney.

■ pandas — ordinale

```
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder
import numpy as np
niveaux = pd.Series(['Bien', 'TB', 'Passable', 'TB', 'Bien'])
ordre = pd.CategoricalDtype(['Passable', 'Assez Bien', 'Bien', 'TB'], ordered=True)
niveaux_ord = niveaux.astype(ordre)
print(niveaux_ord.sort_values().values) # tri respectant l'ordre
# Encodage ordinal pour ML
enc = OrdinalEncoder(categories=[['Passable', 'Assez Bien', 'Bien', 'TB']])
codes = enc.fit_transform(niveaux_ord.values.reshape(-1,1))
print(codes.ravel()) # [2. 3. 0. 3. 2.]
```

3b.3

Intervalle — distances égales, pas de zéro absolu

Ordre et intervalles égaux, mais **pas de zéro absolu réel**. Le zéro est conventionnel. Les additions et soustractions sont valides, les ratios ne le sont pas.

Exemples & règle

- Température en °C : 0°C ≠ absence de chaleur. 30°C / 10°C ≠ 3 (invalide)
 - Année : 2024 – 2020 = 4 ans (valide). 2024 / 2020 (invalide)
 - Score QI : 0 ne signifie pas zéro intelligence
- Opérations autorisées : moyenne, écart-type, test t, ANOVA, Pearson, régression.

3b.4

Ratio — zéro absolu réel, toutes opérations valides

L'échelle la plus complète : ordre, intervalles égaux **et zéro absolu réel**. Toutes les opérations arithmétiques sont valides. C'est l'échelle la plus courante en Data Science.

Exemples & règle

- Salaire en FCFA : 0 FCFA = absence totale de revenu. 600 000 = 2 × 300 000 ✓
 - Âge : 0 an = naissance. 40 ans = 2 × 20 ans ✓
 - Poids, hauteur, distance, nombre de transactions, temps écoulé
- Opérations autorisées : toutes — moyenne, std, ratios, tous tests paramétriques.

Tableau de synthèse des 4 échelles

Échelle	Ordre	Intervalles égaux	Zéro absolu	Exemple	Tests adaptés
Nominale	✗	✗	✗	Genre, Département	Mode, Chi²
Ordinale	✓	✗	✗	Satisfaction, Niveau	Médiane, Spearman
Intervalle	✓	✓	✗	Température °C, QI	Moyenne, test t, Pearson
Ratio	✓	✓	✓	Salaire, Âge, Poids	Tous

SECTION 04

Population et Échantillon

Population	Échantillon
Ensemble complet des individus étudiés. <i>Ex : Tous les habitants d'Abidjan.</i>	Sous-ensemble représentatif de la population. <i>Ex : 1 000 habitants choisis au hasard à Abidjan.</i>

Pourquoi utiliser un échantillon ?

Analyser toute une population est souvent **trop coûteux**, **trop long**, voire **impossible**. Un bon échantillon suffit à estimer les propriétés de la population avec un niveau de confiance contrôlé.

Types d'échantillonnage

La méthode de tirage détermine la qualité et la représentativité de l'échantillon. Un mauvais échantillonnage peut biaiser toute l'analyse, peu importe la sophistication des outils utilisés ensuite.

2.1

Échantillonnage Aléatoire Simple

Chaque individu de la population a la **même probabilité** d'être sélectionné. Le tirage se fait au hasard, sans règle particulière. C'est la méthode de référence, mais elle nécessite une liste complète de la population.

Exemple — Enquête salariale en Côte d'Ivoire

Population : liste des 8 000 employés du secteur bancaire d'Abidjan.

Échantillon : 400 employés tirés au sort dans la liste complète.

→ Chaque employé a $400/8\,000 = 5\%$ de chance d'être sélectionné.

Avantage : simple, sans biais de sélection si le tirage est bien aléatoire.

Limite : nécessite la liste complète de la population (pas toujours disponible).

■ numpy — Aléatoire simple (sans remise)

```
import numpy as np
np.random.seed(42)
population = np.arange(1, 8001)          # IDs des 8 000 employés
# replace=False : un individu ne peut être tiré qu'une seule fois
echantillon = np.random.choice(population, size=400, replace=False)
print(f'Taille : {len(echantillon)} | Ex IDs : {echantillon[:5]}')
```

2.2

Échantillonnage Systématique

On sélectionne un individu tous les **k** individus de la liste, après avoir choisi un point de départ aléatoire. $k = N / n$ (taille population / taille échantillon souhaité).

Exemple — Contrôle qualité en usine

Une usine produit 5 000 pièces par jour. On veut contrôler 100 pièces.

$k = 5\,000 / 100 = 50$ → on prend 1 pièce toutes les 50 produites.

Point de départ aléatoire : pièce n°23. Ensuite : 73, 123, 173, ...

Avantage : rapide, pas besoin de la liste complète.

Limite : risque de biais si la liste a un motif périodique de même pas k.

■ **numpy — Systématique**

```
import numpy as np
N, n = 5000, 100
k = N // n # pas = 50
debut = np.random.randint(0, k) # point de départ aléatoire
indices = np.arange(debut, N, k)
population = np.arange(1, N + 1)
echantillon_sys = population[indices]
print(f'Pas k = {k} Début = {debut} Taille : {len(echantillon_sys)}')
print(f'Premiers indices : {echantillon_sys[:5]}')
```

2.3

Échantillonnage Stratifié

La population est divisée en **sous-groupes homogènes (strates)**, puis un échantillon aléatoire est tiré **dans chaque strate** proportionnellement à sa taille. Garantit la représentation de chaque groupe.

■ **Exemple — Enquête sur les revenus par région**

Population ivoirienne : Abidjan 40% · Bouaké 20% · San-Pédro 15% · Autres 25%

Échantillon souhaité : 1 000 personnes.

→ Abidjan : 400 | Bouaké : 200 | San-Pédro : 150 | Autres : 250

Avantage : représentation garantie de chaque sous-groupe.

Usage : études sociales, sondages politiques, analyse par département.

■ **pandas — Stratifié proportionnel**

```
import pandas as pd
import numpy as np
np.random.seed(42)
df = pd.DataFrame({
    'id' : range(1000),
    'departement' : np.random.choice(['IT', 'RH', 'Finance', 'Ops'],
                                     p=[0.3, 0.2, 0.25, 0.25], size=1000),
    'salaire' : np.random.normal(350_000, 80_000, 1000).astype(int)
})
# 20% dans chaque département (proportionnel)
echantillon_strat = (
    df.groupby('departement', group_keys=False)
      .apply(lambda g: g.sample(frac=0.20, random_state=42))
)
print(df['departement'].value_counts())
print(echantillon_strat['departement'].value_counts())
# → Les proportions sont respectées dans l'échantillon
```

2.4

Échantillonnage par Grappes (Cluster Sampling)

On divise la population en **groupes naturels (grappes)** — villes, quartiers, entreprises — puis on sélectionne aléatoirement **des grappes entières**. Tous les membres des grappes sélectionnées sont enquêtés.

Exemple — Étude scolaire nationale

Population : tous les lycéens de Côte d'Ivoire (grappes = lycées).

On tire au sort 30 lycées sur 300. → On interroge TOUS les élèves de ces 30 lycées.

Avantage : très économique (pas besoin de se déplacer dans tout le pays).

Limite : si les grappes sont homogènes entre elles, l'échantillon est moins précis.

2.5

Échantillonnage de Convenance (Non probabiliste)

On sélectionne les individus les **plus faciles d'accès**, sans tirage aléatoire. Rapide et peu coûteux, mais **non représentatif** de la population globale. Réservé aux études exploratoires préliminaires.

Exemple & Mise en garde

Interroger les premiers passants devant son bureau pour connaître l'opinion nationale.

Sonder uniquement ses contacts LinkedIn pour estimer le salaire moyen des ingénieurs.

→ BIAIS FORT : seuls certains profils sont accessibles.

→ Ne jamais généraliser les résultats à toute la population.

Tableau récapitulatif des méthodes d'échantillonnage

Méthode	Principe	Avantage	Limite	Usage type
Aléatoire simple	Tirage au hasard dans toute la pop.	Sans biais	Liste complète requise	Enquêtes générales
Systématique	1 individu tous les k	Rapide	Biais si motif périodique	Contrôle qualité
Stratifié	Tirage par sous-groupes	Représentatif par groupe	Strates bien définies requises	Études sociales
Par grappes	Groupes naturels entiers	Économique	Moins précis	Études scolaires
Convenance	Individus accessibles	Très rapide	Biais fort, non généralisable	Pilotes exploratoires

■ numpy — comparaison des estimations selon la méthode

```
import numpy as np
np.random.seed(0)
population = np.random.normal(loc=350_000, scale=80_000, size=10_000)
vraie_moyenne = population.mean()
# Aléatoire simple
simple = np.random.choice(population, size=200, replace=False).mean()
# Systématique
k = len(population) // 200
debut = np.random.randint(0, k)
systematique = population[debut:k][:200].mean()
print(f'Vraie moyenne population : {vraie_moyenne:,.0f} FCFA')
print(f'Aléatoire simple : {simple:,.0f} FCFA')
print(f'Systématique : {systematique:,.0f} FCFA')
# → Les méthodes probabilistes estiment correctement la vraie moyenne
```

SECTION 05

Types de Statistiques

Type	Rôle	Exemples d'outils
Statistique descriptive	Décrit et résume les données existantes.	Moyenne, médiane, variance, histogrammes, boîtes à moustaches.
Statistique inférentielle	Tire des conclusions sur la population à partir d'un échantillon.	Tests d'hypothèse, intervalles de confiance, régressions.

Exemple concret

- **Descriptive** : calculer le salaire moyen observé dans un fichier `employees.csv`.
- **Inférentielle** : tester si le salaire moyen de l'entreprise est significativement différent de la moyenne nationale de 300 000 FCFA.

SECTION 06

Mesures de Tendance Centrale

Les mesures de tendance centrale indiquent **où se situe le centre** d'un ensemble de données. Elles répondent à la question : « *Quelle est la valeur typique ?* »

4.1

Moyenne (Mean)

La moyenne est la **somme de toutes les observations divisée par leur nombre**. C'est la mesure la plus utilisée.

$$\bar{x} = (\sum x_i) / n$$

✓ Avantage(s)

- Utilise toutes les données.
- Calcul simple et intuitif.

✗ Inconvénient(s)

- Très sensible aux valeurs aberrantes (outliers).
- Peut être trompeuse si la distribution est asymétrique.

Exemple — Notes d'examen

Notes : 10, 12, 14, 16, 18 → Moyenne = $(10+12+14+16+18) / 5 = 14$

Avec un outlier : 10, 12, 14, 16, 80 → Moyenne = 26.4 (biaisée !)

→ La moyenne monte à 26.4 alors que 4 étudiants sur 5 ont moins de 17.

■ numpy

```
import numpy as np
notes = np.array([10, 12, 14, 16, 18])
print('Moyenne :', np.mean(notes))           # 14.0

# Avec outlier
notes_out = np.array([10, 12, 14, 16, 80])
print('Moyenne avec outlier :', np.mean(notes_out)) # 26.4
```

4.2

Médiane (Median)

La médiane est la **valeur centrale** d'un ensemble de données **triées par ordre croissant**. Si n est pair, c'est la moyenne des deux valeurs centrales.

→ La médiane divise les données en deux moitiés égales.

Exemple — Salaires (FCFA)

Salaires triés : 150 000 · 200 000 · 210 000 · 220 000 · 5 000 000

Médiane = 210 000 (valeur centrale, position 3 sur 5)

Moyenne = 1 156 000 (gonflée par le salaire de 5 000 000 FCFA)

→ La médiane représente MIEUX la majorité des employés ici.

■ numpy

```
salaires = np.array([150_000, 200_000, 210_000, 220_000, 5_000_000])
print('Médiane :', np.median(salaires))      # 210 000
print('Moyenne :', np.mean(salaires))        # 1 156 000 ← biaisée
```

4.3

Mode

Le mode est la **valeur qui apparaît le plus souvent**. Il peut y en avoir plusieurs (distribution bimodale, multimodale). Très utilisé pour les données catégorielles.

Exemple

Données : 10, 10, 10, 12, 14 → Mode = 10 (apparaît 3 fois)

Données : rouge, bleu, bleu, vert, bleu → Mode = bleu

■ scipy.stats

```
from scipy.stats import mode
notes = np.array([10, 10, 10, 12, 14])
res = mode(notes)
print('Mode :', res.mode)          # 10
print('Fréquence :', res.count)    # 3
```

SECTION 07

Mesures de Dispersion

Les mesures de dispersion indiquent à **quel point les données sont étalées** autour du centre. Deux jeux de données peuvent avoir la même moyenne mais des dispersions très différentes.

5.1

Écart Moyen (Mean Absolute Deviation)

L'écart moyen est la **moyenne des distances absolues** de chaque observation par rapport à la moyenne.

$$EM = (\sum |x_i - \bar{x}|) / n$$

Exemple — Classe de terminale

Notes : 10, 11, 12, 13, 14 Moyenne = 12

Distances : $|10-12|=2$, $|11-12|=1$, $|12-12|=0$, $|13-12|=1$, $|14-12|=2$

Écart moyen = $(2+1+0+1+2) / 5 = 1.2$

→ Les élèves se situent en moyenne à 1.2 point de la moyenne.

5.2

Variance

La variance mesure la **dispersion quadratique** des observations autour de la moyenne. On utilise les carrés pour éviter l'annulation des signes et pour pénaliser davantage les grandes erreurs.

$$s^2 = \sum (x_i - \bar{x})^2 / (n - 1) \quad [\text{estimateur non biaisé}]$$

Pourquoi n-1 et non n ? Parce qu'on travaille sur un échantillon. Diviser par n-1 (correction de Bessel) donne un estimateur non biaisé de la variance de la population.

Exemple — Deux entreprises, même moyenne

Entreprise A — Salaires : 490 000 · 500 000 · 510 000 → Moyenne = 500 000

Entreprise B — Salaires : 100 000 · 500 000 · 900 000 → Moyenne = 500 000

Variance A ≈ 100 000 000 (faible : salaires homogènes)

Variance B ≈ 160 000 000 000 (forte : salaires très dispersés)

→ La moyenne seule ne suffit pas. La variance révèle l'inégalité.

■ numpy

```
A = np.array([490_000, 500_000, 510_000])
B = np.array([100_000, 500_000, 900_000])
print('Variance A :', np.var(A, ddof=1))    # ddof=1 → estimateur non biaisé
print('Variance B :', np.var(B, ddof=1))
```

5.3

Écart-type (Standard Deviation)

L'écart-type est la **racine carrée de la variance**. Il a la même unité que les données, ce qui le rend directement interprétable.

$$s = \sqrt{s^2}$$

Interprétation concrète

Moyenne = 100 kg · Écart-type = 10 kg

→ La majorité des observations se situe entre 90 et 110 kg.

Règle empirique (distribution normale) :

- 68% des données sont dans $[x - s, x + s]$
- 95% des données sont dans $[x - 2s, x + 2s]$
- 99.7% des données sont dans $[x - 3s, x + 3s]$

■ numpy

```
data = np.array([490_000, 500_000, 510_000])
print('Écart-type :', np.std(data, ddof=1))

# Impact d'un outlier
data_out = np.append(data, 2_000_000)
print('Std sans outlier :', np.std(data, ddof=1).round(0))
print('Std avec outlier :', np.std(data_out, ddof=1).round(0))
# → L'outlier fait exploser l'écart-type
```


SECTION 08

Loi Normale & Théorème Central Limite

La **loi normale** (ou distribution de Gauss) est la distribution la plus importante en statistique. Le **Théorème Central Limite (TCL)** explique pourquoi elle apparaît partout et justifie l'utilisation des tests paramétriques même sur des données non normales.

6b.1

La Loi Normale

La loi normale est entièrement définie par deux paramètres : la **moyenne** μ (centre) et l'**écart-type** σ (étalement). Sa courbe est en forme de cloche, symétrique autour de μ .

Intervalle	% des données	Exemple ($\mu=170\text{cm}$, $\sigma=10\text{cm}$)
$\mu \pm 1\sigma$	68.27%	160 cm à 180 cm
$\mu \pm 2\sigma$	95.45%	150 cm à 190 cm
$\mu \pm 3\sigma$	99.73%	140 cm à 200 cm (quasi-totalité)

Exemples de phénomènes suivant une loi normale

- Taille des individus d'une même population
 - Erreurs de mesure d'instruments de précision
 - Notes d'examen sur un grand groupe d'étudiants
 - Résidus d'un modèle de régression linéaire
- Vérifier visuellement : histogramme en cloche + droite QQ-plot alignée.

■ scipy.stats — Loi normale

```
import numpy as np
from scipy import stats

# Simulation : tailles de 1000 étudiants
mu, sigma = 170, 10
tailles = np.random.normal(mu, sigma, 1000)

# Probabilités
p_entre_160_180 = stats.norm.cdf(180, mu, sigma) - stats.norm.cdf(160, mu, sigma)
print(f'P(160 ≤ X ≤ 180) = {p_entre_160_180:.2%}')

# Valeur correspondant au percentile 95
p95 = stats.norm.ppf(0.95, mu, sigma)
print(f'95e percentile : {p95:.1f} cm')

# Paramètres estimés depuis un échantillon
mu_est, sigma_est = stats.norm.fit(tailles)
print(f'μ estimé={mu_est:.1f}  σ estimé={sigma_est:.1f}')
```

6b.2

Théorème Central Limite (TCL)

TCL : Quelle que soit la distribution des données originales, la distribution des **moyennes d'échantillons** tend vers une loi normale lorsque la taille d'échantillon n est suffisamment grande ($n \geq 30$ en pratique). C'est pourquoi les tests paramétriques fonctionnent même sur des données non normales.

Taille n	Distribution des moyennes	Implication pratique
$n < 10$	Dépend fortement des données	Tests non paramétriques recommandés
$10 \leq n < 30$	Approximation normale acceptable	Vérifier la normalité (Shapiro-Wilk)
$n \geq 30$	Normale — garanti par le TCL	Tests paramétriques (t, ANOVA) applicables

Exemple concret — Distribution des salaires

Les salaires en Côte d'Ivoire ne suivent PAS une loi normale (asymétrie droite forte).

Mais si on tire 50 échantillons de 100 salaires et qu'on calcule leur moyenne :

→ Ces 50 moyennes suivront une distribution normale (TCL).

→ On peut donc appliquer un test t sur la moyenne, même si les salaires ne sont pas normaux.

■ numpy — Démonstration du TCL

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
# Population très asymétrique (log-normale : revenus)
population = np.random.lognormal(mean=12, sigma=1, size=100_000)
print(f'Skewness population : {population.std():.0f} (très asymétrique)')
# Tirer 1000 échantillons de taille n et calculer leur moyenne
for n in [5, 30, 100]:
    moyennes = [np.random.choice(population, n).mean() for _ in range(1000)]
    sk = abs(np.array(moyennes).std())
    _, p = __import__('scipy').stats.normaltest(moyennes)
    print(f'n={n:3d} → p normalité={p:.3f} {'✓ Normale' if p>0.05 else '■ Pas encore' })
# n=5   → distribution des moyennes non normale
# n=30  → déjà bien approximée
# n=100 → clairement normale (TCL confirmé)
```

SECTION 09

Erreur Standard

L'erreur standard ne mesure **pas** la dispersion des individus. Elle mesure la **précision de la moyenne estimée** : à quel point la moyenne de l'échantillon se rapproche de la moyenne réelle de la population.

$$SE = s / \sqrt{n}$$

Mesure	Ce qu'elle quantifie	Formule
Écart-type (s)	Dispersion des individus autour de la moyenne	$\sqrt{(\sum (x_i - \bar{x})^2) / (n-1)}$
Erreur standard (SE)	Précision de la moyenne estimée	$s / \sqrt{n}$

Exemple — Institut de sondage

Un sondage interroge 100 personnes : $std = 50\,000$ FCFA $\rightarrow SE = 50\,000 / \sqrt{100} = 5\,000$ FCFA

Avec 1 000 personnes : $SE = 50\,000 / \sqrt{1000} \approx 1\,581$ FCFA

→ Plus l'échantillon est grand, plus l'estimation de la moyenne est précise.

→ L'intervalle de confiance à 95% est : $\bar{x} \pm 1.96 \times SE$

■ numpy | scipy.stats

```
from scipy import stats
import numpy as np

salaires = np.random.normal(350_000, 80_000, 200)

se = stats.sem(salaires) # Erreur standard
ic_bas = salaires.mean() - 1.96 * se
ic_haut = salaires.mean() + 1.96 * se

print(f'Moyenne      : {salaires.mean():.0f} FCFA')
print(f'Erreur std   : {se:,.0f} FCFA')
print(f'IC 95%       : [{ic_bas:,.0f} ; {ic_haut:,.0f}]')
```

SECTION 10

Asymétrie & Kurtosis

7.1

Asymétrie (Skewness)

L'asymétrie mesure l'**inclinaison de la distribution** par rapport à une distribution symétrique (en cloche).

Valeur	Forme	Signification
= 0	Symétrique	Moyenne = Médiane = Mode
> 0	Queue à droite ■	Quelques très grandes valeurs (ex : revenus)
< 0	Queue à gauche ■	Quelques très petites valeurs

Exemple réel — Distribution des salaires en Côte d'Ivoire

La majorité des salaires est entre 100 000 et 400 000 FCFA.

Quelques PDG gagnent 5 000 000+ FCFA → queue à droite → skewness > 0.

→ Moyenne > Médiane dans ce cas. La médiane représente mieux la réalité.

7.2

Kurtosis

Le kurtosis mesure l'**épaisseur des queues** de la distribution. Il indique la fréquence des valeurs extrêmes.

Kurtosis	Type	Caractéristique
= 0	Normale (mésokurtique)	Queues standard
> 0	Leptokurtique	Queues épaisses, valeurs extrêmes fréquentes
< 0	Platykurtique	Queues fines, peu de valeurs extrêmes

Exemple réel — Marché boursier

En période de crise financière, les mouvements extrêmes (chutes ou hausses brutales) sont plus fréquents qu'une distribution normale ne le prédit.

→ Le kurtosis des rendements boursiers est souvent > 0 (leptokurtique).

■ **scipy.stats**

```
from scipy import stats
import numpy as np

revenus = np.random.lognormal(mean=12.5, sigma=1, size=1000)
print(f'Skewness : {stats.skew(revenus):.2f}')      # > 0 → asymétrie droite
print(f'Kurtosis : {stats.kurtosis(revenus):.2f}')  # > 0 → queues épaisses

# Test de normalité
stat, p = stats.normaltest(revenus)
print(f'Test normalité p = {p:.4f}')               # < 0.05 → pas normale
```

SECTION 11

Test de Normalité : Shapiro-Wilk

Avant d'appliquer tout test paramétrique (test t, ANOVA, Pearson), il faut vérifier que les données suivent approximativement une loi normale. Le **test de Shapiro-Wilk** est le test de normalité le plus fiable pour des échantillons de taille $n \leq 5\,000$.

H_0 : Les données suivent une loi normale.

H_1 : Les données ne suivent pas une loi normale.

Résultat	Décision	Suite à donner
$p \geq 0.05$	Ne pas rejeter $H_0 \rightarrow$ données normales	Tests paramétriques autorisés (test t, ANOVA, Pearson)
$p < 0.05$	Rejeter $H_0 \rightarrow$ données non normales	Tests non paramétriques (Mann-Whitney, Kruskal-Wallis, Spearman)

Méthodes complémentaires pour évaluer la normalité

Méthode	Description	Avantage
Shapiro-Wilk	Test statistique ($n \leq 5\,000$)	Très précis sur petits échantillons
D'Agostino-Pearson	Basé sur skewness + kurtosis	Robuste sur grands échantillons
Histogramme	Visualisation de la distribution	Intuitif, détecte l'asymétrie
QQ-plot	Points alignés = normalité	Repère les déviations dans les queues

Exemple — Choisir le bon test

Données : salaires de 30 employés d'une PME.

Shapiro-Wilk : $p = 0.021 < 0.05 \rightarrow$ données non normales (asymétrie droite).

$\rightarrow$ Comparer le salaire moyen à 300 000 FCFA : utiliser le test de Wilcoxon (non paramétrique).

$\rightarrow$ Comparer salaires hommes vs femmes : utiliser Mann-Whitney (non paramétrique).

■ **scipy.stats** — Test de Shapiro-Wilk + D'Agostino

```

import numpy as np
from scipy import stats
np.random.seed(42)
# Données normales
donnees_norm = np.random.normal(350_000, 80_000, 50)
# Données asymétriques (log-normale : typique des salaires)
donnees_asym = np.random.lognormal(12.5, 0.8, 50)
def tester_normalite(data, label):
    stat_sw, p_sw = stats.shapiro(data)
    stat_da, p_da = stats.normaltest(data) # D'Agostino-Pearson
    print(f'{label}')
    print(f' Shapiro-Wilk      : W={stat_sw:.3f}  p={p_sw:.4f} → {'NORMALE ✓' if p_sw >= 0.05 else 'NON normale ✗'})
    print(f' D'Agostino          : stat={stat_da:.3f}  p={p_da:.4f} → {'NORMALE ✓' if p_da >= 0.05 else 'NON normale ✗'})
    print(f' Skewness            : {stats.skew(data):.2f}  Kurtosis : {stats.kurtosis(data):.2f}')
    print()
tester_normalite(donnees_norm, 'Salaires simulés normaux')
tester_normalite(donnees_asym, 'Salaires log-normaux (réalistes)')
# Arbre de décision après test
p = stats.shapiro(donnees_asym)[1]
if p >= 0.05:
    print('→ Données normales : utiliser test t, ANOVA, Pearson')
else:
    print('→ Données non normales : utiliser Mann-Whitney, Kruskal-Wallis, Spearman')

```

SECTION 12

Covariance

La **covariance** mesure dans quelle mesure deux variables varient **ensemble**. C'est le fondement mathématique de la corrélation. Comprendre la covariance permet de comprendre pourquoi et comment les variables sont liées.

$$\text{Cov}(X, Y) = \sum (x_i - \bar{x})(y_i - \bar{y}) / (n - 1)$$

Signe de Cov(X,Y)	Signification	Exemple
Cov > 0	Quand X augmente, Y tend à augmenter aussi	Ancienneté ↑ → Salaire ↑
Cov < 0	Quand X augmente, Y tend à diminuer	Absentéisme ↑ → Performance ↓
Cov ≈ 0	Aucune relation linéaire entre X et Y	Pointure ↔ Salaire

Limite de la covariance

La covariance dépend des **unités de mesure** des variables. Cov(Salaire en FCFA, Âge en années) = 1 500 000 et Cov(Salaire en milliers FCFA, Âge) = 1 500 mesurent la même relation mais donnent des valeurs très différentes. C'est pourquoi on la **normalise** pour obtenir la corrélation :

$$r = \text{Cov}(X, Y) / (s_X \times s_Y)$$

Exemple concret — Ancienneté & Salaire

Employé 1 : ancienneté = 2 ans, salaire = 250 000 FCFA

Employé 2 : ancienneté = 8 ans, salaire = 420 000 FCFA

Employé 3 : ancienneté = 15 ans, salaire = 600 000 FCFA

Cov > 0 → Plus on a d'ancienneté, plus le salaire est élevé.

→ La corrélation de Pearson normalisera cette valeur entre -1 et +1.

■ numpy | pandas — Covariance

```
import numpy as np
import pandas as pd

anciennete = np.array([2, 5, 8, 10, 15, 18, 20])
salaire = np.array([250, 310, 380, 420, 520, 570, 620]) # k FCFA

# Covariance manuelle
cov_manuelle = np.sum(
    (anciennete - anciennete.mean()) * (salaire - salaire.mean())
) / (len(anciennete) - 1)
print(f'Covariance manuelle : {cov_manuelle:.2f}')

# NumPy (matrice de covariance)
cov_matrix = np.cov(anciennete, salaire)
print('Matrice de covariance :')
print(cov_matrix.round(2))
print(f'Cov(X,Y) = {cov_matrix[0,1]:.2f}')

# Pandas (toutes les paires d'un DataFrame)
df = pd.DataFrame({'anciennete': anciennete, 'salaire': salaire})
print(' Matrice pandas :')
print(df.cov().round(2))

# De la covariance à la corrélation
r = cov_matrix[0,1] / (anciennete.std(ddof=1) * salaire.std(ddof=1))
print(f' Corrélation de Pearson : r = {r:.3f}')

# Vérification
print(f'Via np.corrcoef      : r = {np.corrcoef(anciennete, salaire)[0,1]:.3f}')
```


SECTION 13

Corrélation

La corrélation mesure la **force et la direction du lien** entre deux variables. Elle est comprise entre -1 et $+1$.
Attention : corrélation $\neq$ causalité.

8.1

Corrélation de Pearson

Mesure les **relations linéaires**. Suppose que les données suivent approximativement une distribution normale.
 Sensible aux outliers.

Valeur de r	Interprétation
+1.0	Corrélation linéaire parfaite positive
+0.7 à +0.9	Corrélation forte positive
+0.4 à +0.6	Corrélation modérée positive
0	Aucune corrélation linéaire
-0.7 à -0.9	Corrélation forte négative
-1.0	Corrélation linéaire parfaite négative

8.2

Corrélation de Spearman

Mesure les **relations monotones** (pas nécessairement linéaires). Basée sur les rangs. **Robuste aux outliers**.
 Adaptée aux données ordinales ou non normales.

Exemples réels

- *Pearson* : température et consommation d'électricité (relation linéaire).
- *Spearman* : classement des étudiants et classement des salaires (données de rang).
- Si $r_{\text{Pearson}} \neq r_{\text{Spearman}} \rightarrow$ des outliers influencent fortement la corrélation.

■ `scipy.stats`

```
from scipy.stats import pearsonr, spearmanr
import numpy as np

age      = np.array([22, 25, 30, 35, 40, 45, 50, 55])
salaire = np.array([150, 200, 280, 350, 420, 480, 530, 600]) # k FCFA

r_p, p_p = pearsonr(age, salaire)
r_s, p_s = spearmanr(age, salaire)

print(f'Pearson   : r = {r_p:.3f}  p = {p_p:.4f}')
print(f'Spearman  : r = {r_s:.3f}  p = {p_s:.4f}')

if p_p < 0.05:
    print('→ Corrélation significative au seuil 5%')
```

SECTION 14

Visualisation Statistique

Un graphique bien choisi révèle en quelques secondes ce que des heures de calcul ne montreraient pas. Le choix du graphique dépend du **type de données** et de la **question posée**.

Graphique	Type de données	Question posée	Exemple
Histogramme	Quantitative continue	Quelle est la distribution ?	Distribution des salaires
Boxplot	Quantitative	Y a-t-il des outliers ? Comparer groupes ?	Salaire par département
Barres	Qualitative	Quelle catégorie domine ?	Nb employés par département
Scatter plot	2 quantitatives	Y a-t-il une relation ?	Âge vs Salaire
Heatmap corrél.	Plusieurs quantitatives	Quelles variables sont liées ?	Matrice de corrélation
QQ-plot	Quantitative	La distribution est-elle normale ?	Résidus d'une régression
Courbe temporelle	Série temporelle	Quelle est la tendance ?	Ventes sur 12 mois

■ matplotlib | scipy — Dashboard de visualisation complet

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

np.random.seed(42)
n = 200
df = pd.DataFrame({
    'age' : np.random.randint(22, 60, n),
    'salaire' : np.random.lognormal(12.5, 0.5, n).astype(int),
    'anciennete' : np.random.randint(0, 25, n),
    'departement': np.random.choice(['IT', 'RH', 'Finance', 'Ops'], n),
})

fig, axes = plt.subplots(2, 3, figsize=(14, 9))
fig.suptitle('Dashboard Statistique – Données Employés',
             fontsize=13, fontweight='bold', color='#1A237E')

# 1. Histogramme – distribution des salaires
axes[0,0].hist(df['salaire'], bins=25, color='#1A237E', alpha=0.8,
               edgecolor='white')
axes[0,0].axvline(df['salaire'].mean(), color='#F57C00', lw=2, label='Moyenne')
axes[0,0].axvline(df['salaire'].median(), color='red', lw=2, label='Médiane')
axes[0,0].set_title('Distribution des Salaires')
axes[0,0].legend(fontsize=8)

# 2. Boxplot – salaires par département
depts = ['IT', 'RH', 'Finance', 'Ops']
data_bp = [df[df['departement']==d]['salaire'].values for d in depts]
bp = axes[0,1].boxplot(data_bp, labels=depts, patch_artist=True)
for patch, color in zip(bp['boxes'], ['#1A237E', '#283593', '#3949AB', '#5C6BC0']):
    patch.set_facecolor(color)
    patch.set_alpha(0.7)
axes[0,1].set_title('Salaires par Département')

# 3. Scatter plot – âge vs salaire
axes[0,2].scatter(df['age'], df['salaire'], alpha=0.5,
                  color='#F57C00', s=25)
m, b = np.polyfit(df['age'], df['salaire'], 1)
axes[0,2].plot(df['age'].sort_values(),
               m*df['age'].sort_values() + b, '--', color='#1A237E')
axes[0,2].set_title('Âge vs Salaire + Tendance')

# 4. Barres – nb employés par département
counts = df['departement'].value_counts()
axes[1,0].bar(counts.index, counts.values, color='#1A237E', alpha=0.8)
axes[1,0].set_title('Effectifs par Département')

# 5. Heatmap corrélation
corr = df[['age', 'salaire', 'anciennete']].corr()
im = axes[1,1].imshow(corr, cmap='Blues', vmin=-1, vmax=1)
axes[1,1].set_xticks(range(3))
axes[1,1].set_yticks(range(3))
axes[1,1].set_xticklabels(corr.columns, fontsize=8)
axes[1,1].set_yticklabels(corr.columns, fontsize=8)
for i in range(3):

```

```
for j in range(3):
    axes[1,1].text(j, i, f'{corr.iloc[i,j]:.2f}',
                  ha='center', va='center', fontsize=9, color='white')
axes[1,1].set_title('Heatmap Corrélations')
# 6. QQ-plot – normalité des salaires
stats.probplot(df['salaire'], dist='norm', plot=axes[1,2])
axes[1,2].set_title('QQ-plot (Normalité)')
axes[1,2].get_lines()[0].set(color='#F57C00', markersize=3)
plt.tight_layout()
plt.savefig('dashboard_stats.png', dpi=150, bbox_inches='tight')
print('Dashboard sauvegardé ✓')
```

SECTION 15

Tests d'Hypothèse

Un test d'hypothèse permet de déterminer si un résultat observé est dû au **hasard** ou à un **effet réel**, avec un niveau de risque contrôlé.

H₀ — Hypothèse nulle

L'affirmation de départ. On suppose qu'il n'y a pas d'effet réel.

Ex : Le médicament n'a aucun effet sur les patients.

Ex : Le salaire moyen = 300 000 FCFA.

H₁ — Hypothèse alternative

Ce qu'on cherche à démontrer. Il y a un effet réel.

Ex : Le médicament améliore l'état du patient.

Ex : Le salaire moyen $\neq$ 300 000 FCFA.

SECTION 16

p-value

La **p-value** est la probabilité d'obtenir un résultat aussi extrême (ou plus) que celui observé, **en supposant que H_0 est vraie**.

→ Une p-value faible signifie que le résultat est très improbable sous H_0 .

p-value	Décision	Signification
$p < 0.01$	Rejet fort de H_0	Résultat très significatif
$p < 0.05$	Rejet de H_0	Résultat significatif (seuil standard)
$0.05 \leq p < 0.10$	Zone grise	Tendance, à interpréter avec prudence
$p \geq 0.10$	Pas de rejet de H_0	Pas de preuve suffisante contre H_0

Attention : p-value $\neq$ probabilité que H_0 est vraie

$p = 0.03$ signifie : 'Si H_0 était vraie, on observerait ce résultat dans 3% des cas.'

Cela NE signifie PAS que H_0 a 3% de chances d'être vraie.

→ Toujours interpréter la p-value en contexte, avec une taille d'échantillon suffisante.

SECTION 16b

Intervalle de Confiance

Un **intervalle de confiance (IC)** est une plage de valeurs dans laquelle on estime que le paramètre de la population (ex : la vraie moyenne) se trouve avec un certain niveau de probabilité (ex : 95%).

$$IC_{95\%} = x \pm 1.96 \times SE = x \pm 1.96 \times (s / \sqrt{n})$$

Niveau de confiance	Coefficient z	Interprétation
90%	1.645	1 chance sur 10 que la vraie valeur soit hors de l'intervalle
95%	1.960	Standard en science. 5% de risque d'erreur
99%	2.576	Très conservateur. Intervalle plus large

Exemple — Salaire moyen estimé

Échantillon : $n = 150$ employés. $x = 320\,000$ FCFA. $s = 70\,000$ FCFA.

$SE = 70\,000 / \sqrt{150} \approx 5\,715$ FCFA

$IC\ 95\% = [320\,000 - 1.96 \times 5\,715 ; 320\,000 + 1.96 \times 5\,715]$

$IC\ 95\% = [308\,800\text{ FCFA} ; 331\,200\text{ FCFA}]$

→ On est sûr à 95% que le salaire moyen réel est entre 308 800 et 331 200 FCFA.

■ scipy.stats — Intervalle de confiance

```
import numpy as np
from scipy import stats

np.random.seed(42)
salaires = np.random.normal(320_000, 70_000, 150)

n = len(salaires)
moyenne = salaries.mean()
se = stats.sem(salaires)

# IC à 95% via la loi t de Student (plus précis que z pour petit n)
ic_bas, ic_haut = stats.t.interval(
    confidence=0.95, df=n-1, loc=moyenne, scale=se)

print(f'Moyenne : {moyenne:,.0f} FCFA')
print(f'SE : {se:,.0f} FCFA')
print(f'IC 95% : [{ic_bas:,.0f} ; {ic_haut:,.0f}] FCFA')

# IC à 99%
ic99_b, ic99_h = stats.t.interval(
    confidence=0.99, df=n-1, loc=moyenne, scale=se)
print(f'IC 99% : [{ic99_b:,.0f} ; {ic99_h:,.0f}] FCFA')

# → IC 99% plus large : plus de sécurité, moins de précision
```

SECTION 17

Test t de Student

Le test t permet de **comparer des moyennes** pour décider si une différence observée est statistiquement significative ou due au hasard.

$$t = (x - \mu_0) / (s / \sqrt{n})$$

Le t-score mesure **combien d'erreurs standards séparent la moyenne observée de la moyenne théorique** μ_0 . Plus $|t|$ est grand, plus l'écart est significatif.

Type de test	Usage
Test à 1 échantillon	Comparer la moyenne d'un groupe à une valeur de référence
Test à 2 échantillons indépendants	Comparer les moyennes de deux groupes distincts
Test apparié	Comparer les moyennes avant/après sur le même groupe

Exemple — Salaire moyen des employés

H_0 : Le salaire moyen = 300 000 FCFA (valeur nationale)

H_1 : Le salaire moyen $\neq$ 300 000 FCFA

Si $t = 2.8$ et $p = 0.02 \rightarrow p < 0.05 \rightarrow$ on rejette H_0

$\rightarrow$ Le salaire moyen de cette entreprise est significativement différent de 300 000 FCFA.

■ scipy.stats

```
from scipy.stats import ttest_1samp, ttest_ind
import numpy as np

salaires = np.array([280_000, 310_000, 295_000, 320_000, 285_000,
                    340_000, 260_000, 315_000, 298_000, 305_000])

# Test à 1 échantillon
t, p = ttest_1samp(salaires, popmean=300_000)
print(f't-score : {t:.4f} | p-value : {p:.4f}')
print('→ H0 rejetée' if p < 0.05 else '→ H0 non rejetée')

# Test à 2 échantillons : hommes vs femmes
hommes = np.array([350_000, 380_000, 320_000, 400_000, 360_000])
femmes = np.array([290_000, 310_000, 280_000, 305_000, 295_000])
t2, p2 = ttest_ind(hommes, femmes)
print(f'Écart H/F : p = {p2:.4f}')
```


SECTION 18

ANOVA — Analyse de la Variance

L'ANOVA (Analysis of Variance) teste si les moyennes de **3 groupes ou plus** sont significativement différentes. C'est l'extension du test t pour plus de 2 groupes.

Pourquoi ne pas faire plusieurs tests t ?

Comparer 4 groupes avec des tests t 2 à 2 donnerait 6 comparaisons. À chaque test on prend un risque $\alpha = 5\%$. Le risque global d'erreur grimpe à $1 - (1-0.05)^6 \approx 26\%$! L'ANOVA résout ce problème en un seul test.

H_0 (ANOVA) : $\mu_1 = \mu_2 = \mu_3 = \dots = \mu_k$ (toutes les moyennes sont égales)

H_1 (ANOVA) : Au moins une moyenne est différente des autres

Statistique	Signification
F-score	Ratio variance inter-groupes / variance intra-groupes. Grand F → groupes différents
p-value	Si $p < 0.05$: au moins un groupe est différent des autres
Post-hoc (Tukey)	Test complémentaire pour identifier QUELS groupes différent

Exemple — Salaire moyen par département

H_0 : Le salaire moyen est identique dans les 4 départements (IT, RH, Finance, Ops).

H_1 : Au moins un département a un salaire moyen différent.

Si $F = 8.3$ et $p = 0.0002 < 0.05 \rightarrow H_0$ rejetée.

→ Il existe au moins un département avec un salaire significativement différent.

→ Lancer un test post-hoc (Tukey HSD) pour identifier lequel.

■ scipy.stats | statsmodels — ANOVA à 1 facteur + Tukey

```
import numpy as np
from scipy import stats
from statsmodels.stats.multicomp import pairwise_tukeyhsd

np.random.seed(0)
it      = np.random.normal(420_000, 60_000, 40)
rh      = np.random.normal(310_000, 50_000, 35)
finance = np.random.normal(480_000, 70_000, 45)
ops     = np.random.normal(350_000, 55_000, 38)

# ANOVA à 1 facteur
F, p = stats.f_oneway(it, rh, finance, ops)
print(f'F-score : {F:.2f}    p-value : {p:.4f}')
print('→ H0 rejetée' if p < 0.05 else '→ H0 non rejetée')

# Test post-hoc Tukey HSD (si ANOVA significative)
if p < 0.05:
    import pandas as pd
    salaires = np.concatenate([it, rh, finance, ops])
    groupes  = ['IT']*40 + ['RH']*35 + ['Finance']*45 + ['Ops']*38
    tukey = pairwise_tukeyhsd(salaires, groupes, alpha=0.05)
    print(tukey)
```

SECTION 19

Test du Chi² — Variables Catégorielles

Le **test du Chi²** teste l'**indépendance entre deux variables catégorielles**. Il compare les fréquences observées aux fréquences attendues si les variables étaient indépendantes.

$$\chi^2 = \sum [(O - E)^2 / E]$$

O = fréquence observée | E = fréquence attendue sous H₀

Hypothèse	Signification
H ₀	Les deux variables sont indépendantes (aucun lien entre elles)
H ₁	Les deux variables sont dépendantes (il existe un lien)
p < 0.05	→ Rejeter H ₀ : les variables sont liées

Exemple — Genre et type de contrat

Question : Le genre influence-t-il le type de contrat (CDI / CDD / Stage) ?

H₀ : Genre et type de contrat sont indépendants.

H₁ : Il existe un lien entre genre et type de contrat.

Si p = 0.008 < 0.05 → on rejette H₀ : il y a un lien statistiquement significatif.

■ scipy.stats | pandas — Chi²

```
import pandas as pd
import numpy as np
from scipy.stats import chi2_contingency

np.random.seed(7)
n = 200
df = pd.DataFrame({
    'genre' : np.random.choice(['Homme', 'Femme'], n),
    'contrat' : np.random.choice(['CDI', 'CDD', 'Stage'],
                                p=[0.5, 0.3, 0.2], size=n)
})

# Tableau de contingence
contingence = pd.crosstab(df['genre'], df['contrat'])
print(contingence)

# Test Chi2
chi2, p, dof, attendu = chi2_contingency(contingence)
print(f' Chi2 = {chi2:.3f}   ddl = {dof}   p-value = {p:.4f}')
print('→ Lien significatif' if p < 0.05 else '→ Variables indépendantes')

# Fréquences attendues sous H0
print(' Fréquences attendues :', attendu.round(1))
```

SECTION 20

Tests Non Paramétriques

Les tests non paramétriques ne supposent **aucune distribution particulière** des données. Ils travaillent sur les **rangs** plutôt que sur les valeurs brutes. À utiliser quand Shapiro-Wilk indique $p < 0.05$, ou pour les données ordinales et les petits échantillons.

Test non paramétrique	Équivalent paramétrique	Quand l'utiliser
Mann-Whitney U	Test t (2 groupes indép.)	Comparer 2 groupes, données non normales
Wilcoxon signé	Test t apparié	Comparer avant/après sur mêmes individus
Kruskal-Wallis	ANOVA à 1 facteur	Comparer 3+ groupes, données non normales
Spearman	Pearson	Corrélation sur données ordinales ou non normales
Chi ² (vu en 11d)	—	Indépendance entre 2 variables catégorielles

11e.1

Test de Mann-Whitney U

Compare les distributions de **deux groupes indépendants**. Teste si l'un des groupes tend à avoir des valeurs systématiquement plus élevées. Robuste aux outliers et ne suppose pas la normalité.

H_0 : Les deux groupes ont la même distribution.

H_1 : Les distributions sont différentes (l'un est décalé par rapport à l'autre).

Exemple — Salaire hommes vs femmes (distribution non normale)

Shapiro-Wilk sur les salaires $\rightarrow p = 0.003 \rightarrow$ données non normales.

$\rightarrow$ On ne peut pas utiliser le test t. On utilise Mann-Whitney.

Mann-Whitney : $p = 0.008 < 0.05 \rightarrow H_0$ rejetée.

$\rightarrow$ Il existe une différence significative de salaire entre hommes et femmes.

■ **scipy.stats — Mann-Whitney U**

```
import numpy as np
from scipy import stats
np.random.seed(42)
# Salaires non normaux (log-normaux)
hommes = np.random.lognormal(12.8, 0.6, 60)
femmes = np.random.lognormal(12.5, 0.6, 55)
# Vérifier la normalité d'abord
_, p_h = stats.shapiro(hommes)
_, p_f = stats.shapiro(femmes)
print(f'Shapiro hommes : p={p_h:.4f}  femmes : p={p_f:.4f}')
# Test approprié selon normalité
if p_h < 0.05 or p_f < 0.05:
    stat, p = stats.mannwhitneyu(hommes, femmes, alternative='two-sided')
    print(f'→ Mann-Whitney U : stat={stat:.0f}  p={p:.4f}')
else:
    stat, p = stats.ttest_ind(hommes, femmes)
    print(f'→ Test t : stat={stat:.3f}  p={p:.4f}')
print('Différence significative ✓' if p < 0.05 else 'Pas de différence ✗')
# Médiane (plus robuste que la moyenne pour données non normales)
print(f'Médiane hommes : {np.median(hommes):.0f} FCFA')
print(f'Médiane femmes : {np.median(femmes):.0f} FCFA')
```

11e.2

Test de Kruskal-Wallis

Extension de Mann-Whitney pour **3 groupes ou plus**. Alternative non paramétrique à l'ANOVA. Comme pour l'ANOVA, si le test est significatif, utiliser un test post-hoc (Dunn) pour identifier quels groupes diffèrent.

H_0 : Tous les groupes ont la même distribution.

H_1 : Au moins un groupe a une distribution différente.

Exemple — Satisfaction client par région (données ordinales)

Scores de satisfaction (1 à 5) : Abidjan, Bouaké, Yamoussoukro.

Données ordinales → non normales → test de Kruskal-Wallis.

$p = 0.03 < 0.05 \rightarrow H_0$ rejetée.

→ La satisfaction varie significativement selon la région. → Lancer le test de Dunn.

■ `scipy.stats` | `scikit_posthocs` — Kruskal-Wallis + Dunn

```

import numpy as np
from scipy import stats
np.random.seed(0)
# Scores de satisfaction ordinaux (1-5) par département
it      = np.random.choice([3,4,5], 40, p=[0.2,0.4,0.4])
rh      = np.random.choice([2,3,4], 35, p=[0.3,0.4,0.3])
finance = np.random.choice([1,2,3], 45, p=[0.3,0.4,0.3])
ops     = np.random.choice([3,4,5], 38, p=[0.3,0.4,0.3])
# Test de Kruskal-Wallis
H, p = stats.kruskal(it, rh, finance, ops)
print(f'Kruskal-Wallis : H={H:.3f}  p={p:.4f}')
if p < 0.05:
    print('→ H■ rejetée : au moins un département diffère.')
    print('Médianes :',
          {d: np.median(g) for d, g in
           zip(['IT', 'RH', 'Finance', 'Ops'], [it, rh, finance, ops])})
# Post-hoc : comparaisons 2 à 2
for (n1,g1), (n2,g2) in [
    (('IT',it),('Finance',finance)),
    (('RH',rh),('Finance',finance)),
    (('IT',it),('RH',rh)),
]:
    _, p12 = stats.mannwhitneyu(g1, g2, alternative='two-sided')
    print(f' {n1} vs {n2} : p={p12:.4f} {'*' if p12<0.05 else 'ns'})

```

SECTION 21

Z-score, IQR et Détection d'Outliers

Le Z-score mesure combien d'écart-types une valeur s'éloigne de la moyenne. Il standardise les données pour permettre des comparaisons entre variables d'unités différentes.

$$z = (x - \bar{x}) / s$$

z-score	Interprétation
$ z < 2$	Valeur normale, dans la zone courante
$2 \leq z < 3$	Valeur atypique, à surveiller
$ z \geq 3$	Outlier — valeur très probablement aberrante
$ z \geq 4$	Outlier sévère — très probablement une erreur

Exemple — Capteurs IoT d'une usine

Températures : 22, 23, 21, 24, 22, 23, -10, 25, 22, 100, 23 °C

$z(-10) \approx -4.2 \rightarrow$ outlier sévère (capteur défectueux ?)

$z(100) \approx +5.8 \rightarrow$ outlier sévère (surchauffe critique ?)

$\rightarrow$ Ces valeurs doivent être vérifiées avant toute analyse.

■ scipy.stats | numpy

```
from scipy import stats
import numpy as np

temperatures = np.array([22, 23, 21, 24, 22, 23, -10, 25, 22, 100, 23])
z_scores = stats.zscore(temperatures)
print('Z-scores :', z_scores.round(2))

seuil = 3
masque_propre = np.abs(z_scores) <= seuil
masque_outlier = np.abs(z_scores) > seuil

print('Outliers      :', temperatures[mask_outlier])    # [-10, 100]
print('Données propres :', temperatures[mask_propre])
print('Nb outliers    :', mask_outlier.sum())
```

12.2

Méthode IQR (Interquartile Range)

L'IQR (écart interquartile) est la différence entre le 3^e quartile (Q3) et le 1^{er} quartile (Q1). Il représente l'étendue des 50% centraux des données et constitue une mesure **robuste** de la dispersion, insensible aux outliers.

$$IQR = Q_3 - Q_1$$

Règle des clôtures de Tukey

On définit deux bornes au-delà desquelles une valeur est considérée comme aberrante :

Borne	Formule	Interprétation
Limite basse	$Q1 - 1.5 \times IQR$	Toute valeur en dessous = outlier
Limite haute	$Q3 + 1.5 \times IQR$	Toute valeur au-dessus = outlier
Limite extrême basse	$Q1 - 3.0 \times IQR$	Outlier sévère (extrême)
Limite extrême haute	$Q3 + 3.0 \times IQR$	Outlier sévère (extrême)

Exemple concret — Salaires d'une PME (FCFA)

Salaires triés : 180 000 · 220 000 · 250 000 · 270 000 · 300 000 · 320 000 · 350 000 · 380 000 · 420 000 · 2 500 000

$Q1 = 250\,000$ | $Q3 = 380\,000$ | $IQR = 380\,000 - 250\,000 = 130\,000$

Limite haute = $380\,000 + 1.5 \times 130\,000 = 575\,000$ FCFA

→ Le salaire de 2 500 000 FCFA est un outlier (PDG ou erreur de saisie).

Avantage vs Z-score : l'IQR ne suppose PAS une distribution normale.

Z-score	VS	IQR — Tukey
$z = (x - x_{\text{moy}}) / s$ Distance en nombre d'écarts-types à la moyenne.	Formule	$IQR = Q3 - Q1$ Bornes : $Q1 - 1.5 \times IQR$ et $Q3 + 1.5 \times IQR$
Suppose une distribution approximativement normale .	Hypothèse	Aucune hypothèse sur la distribution. Fonctionne sur toute forme de données.
Sensible aux outliers : un seul extrême modifie x_{moy} et s , masquant d'autres outliers.	Robustesse	Robuste : $Q1$ et $Q3$ ne sont pas affectés par les valeurs extrêmes.
$ z > 2 \rightarrow$ atypique $ z > 3 \rightarrow$ outlier $ z > 4 \rightarrow$ outlier sévère	Seuil	$x > Q3 + 1.5 \times IQR \rightarrow$ outlier $x > Q3 + 3.0 \times IQR \rightarrow$ outlier sévère
Continues et normales : temps, scores, capteurs IoT, erreurs de mesure.	Données adaptées	Asymétriques ou inconnues : salaires, revenus, prix, temps de traitement.
Indique combien d'écarts-types une valeur s'éloigne du centre.	Interprétation	Délimite une zone acceptable basée sur les 50% centraux des données.
Histogramme + courbe normale. Repérage visuel des queues de distribution.	Visualisation	Boxplot (boîte à moustaches) : les points hors moustaches = outliers IQR.
Si beaucoup d'outliers : x_{moy} et s biaisés $\rightarrow$ z-score sous-estime les outliers.	Point faible	Peut signaler une valeur légitime en distribution très asymétrique $\rightarrow$ ajuster à $2.0 \times IQR$.
✓ Normalité vérifiée (Shapiro-Wilk) ✓ Petits datasets propres ✓ Standardisation pour ML	Quand choisir ?	✓ Distribution inconnue ou asymétrique ✓ Données financières, sociales ✓ Première détection rapide

SECTION 22

Chargement des Données avec Pandas

Format	Fonction Pandas	Options clés
CSV	pd.read_csv('fichier.csv')	sep, encoding, parse_dates, index_col
Excel	pd.read_excel('fichier.xlsx')	sheet_name, header, usecols
JSON	pd.read_json('fichier.json')	orient, lines
SQL	pd.read_sql(query, con)	connexion SQLAlchemy ou sqlite3

■ pandas — chargement et exploration initiale

```
import pandas as pd
import numpy as np

# Chargement
df = pd.read_csv('employees.csv', sep=',', encoding='utf-8',
                 parse_dates=['date_embauche'])

# Dimensions
print('Shape :', df.shape)          # (nb_lignes, nb_colonnes)

# Aperçu
print(df.head(5))                   # 5 premières lignes
print(df.dtypes)                    # types de colonnes

# Statistiques descriptives
print(df.describe().round(0))        # count, mean, std, min, Q1, Q2, Q3, max

# Valeurs manquantes
print(df.isnull().sum())             # NaN par colonne
print((df.isnull().mean()*100).round(1)) # % de NaN
```

SECTION 23

Nettoyage des Données

Règle d'or : Garbage In → Garbage Out. Un modèle ou une analyse sur données non nettoyées donnera des résultats faux, peu importe la sophistication de l'algorithme.

■ pandas — pipeline de nettoyage complet

```
import pandas as pd
import numpy as np

df = pd.read_csv('employees.csv')

# 1. Supprimer les doublons
print('Doublons avant :', df.duplicated().sum())
df = df.drop_duplicates()

# 2. Uniformiser les noms de colonnes
df.columns = df.columns.str.lower().str.strip().str.replace(' ', '_')

# 3. Corriger les types
df['salaire'] = pd.to_numeric(df['salaire'], errors='coerce')
df['date_embauche'] = pd.to_datetime(df['date_embauche'], errors='coerce')

# 4. Traiter les valeurs manquantes
df['salaire'] = df['salaire'].fillna(df['salaire'].median())
df['departement'] = df['departement'].fillna('Inconnu')

# 5. Nettoyer les chaînes de caractères
df['nom'] = df['nom'].str.strip().str.title() # 'KOHOU' → 'Kohou'

# 6. Supprimer les valeurs incohérentes
df = df[(df['age'].between(18, 65)) & (df['salaire'] > 0)]

print('Dataset propre :', df.shape)
```

SECTION 24

Arbre de Décision des Tests Statistiques

Face à un jeu de données, la question clé est : **quel test utiliser ?** Cet arbre de décision guide le choix selon le type de données, le nombre de groupes et la normalité des données.

Question	Condition	→ Test recommandé	Bibliothèque
Comparer 1 moyenne à une valeur de référence	Données normales (Shapiro $p \geq 0.05$)	Test t à 1 échantillon <code>ttest_1samp()</code>	scipy.stats
Comparer 1 moyenne à une valeur de référence	Données NON normales	Test de Wilcoxon <code>wilcoxon()</code>	scipy.stats
Comparer 2 groupes indépendants	Données normales	Test t indépendant <code>ttest_ind()</code>	scipy.stats
Comparer 2 groupes indépendants	Données NON normales	Mann-Whitney U <code>mannwhitneyu()</code>	scipy.stats
Comparer avant/après (même groupe)	Données normales	Test t apparié <code>ttest_rel()</code>	scipy.stats
Comparer avant/après (même groupe)	Données NON normales	Wilcoxon signé <code>wilcoxon()</code>	scipy.stats
Comparer 3 groupes ou plus	Données normales	ANOVA + Tukey HSD <code>f_oneway()</code>	scipy.stats
Comparer 3 groupes ou plus	Données NON normales	Kruskal-Wallis <code>kruskal()</code>	scipy.stats
Mesurer la relation entre 2 variables numériques	Données normales (linéaire)	Pearson + Covariance <code>pearsonr()</code>	scipy.stats
Mesurer la relation entre 2 variables numériques	Données NON normales ou ordinales	Spearman <code>spearmanr()</code>	scipy.stats
Tester l'indépendance entre 2 variables catégorielles	Données de comptage (tableau contingence)	Chi ² (χ^2) <code>chi2_contingency()</code>	scipy.stats
Détecter les valeurs aberrantes	Distribution normale	Z-score $ z > 3 \rightarrow$ outlier	scipy.stats
Détecter les valeurs aberrantes	Distribution quelconque / asymétrique	IQR Tukey $x > Q3 + 1.5 \times IQR \rightarrow$ outlier	numpy

Les 3 questions à se poser avant tout test

1. Quel est mon objectif ?	2. Combien de groupes ?	3. Données normales ?
Comparer des moyennes ? Mesurer une relation ? Tester l'indépendance ?	1 groupe $\rightarrow$ test 1 éch. 2 groupes $\rightarrow$ t ou Mann-Whitney 3+ $\rightarrow$ ANOVA ou Kruskal	Shapiro-Wilk $p \geq 0.05 ? \rightarrow$ OUI : test paramétrique $\rightarrow$ NON : non paramétrique

SECTION 25

Pipeline Complet d'Analyse de Données

Un pipeline d'analyse de données suit toujours les mêmes étapes dans l'ordre. Sauter une étape compromet la fiabilité des conclusions.

1	COLLECTER	Charger les données — read_csv / read_excel / SQL / API
2	EXPLORER	Comprendre la structure — shape, dtypes, head, describe, isnull
3	NETTOYER	Traiter les problèmes — Doublons, NaN, types, outliers
4	DÉCRIRE	Statistiques descriptives — Moyenne, médiane, std, quartiles, mode
5	VISUALISER	Graphiques exploratoires — Histogramme, boxplot, scatter, heatmap
6	ANALYSER	Tests statistiques — Shapiro → choix test → p-value → décision
7	CONCLURE	Répondre à la question métier — Insights actionnables + limites

■ Pipeline complet — pandas · numpy · scipy · matplotlib

[illegible]

```
print(f'Salaire médian : {df['salaire'].median():.0f} FCFA')
print(f'Corrélation âge : r={r:.2f} ({'sig.' if p_r<0.05 else 'non sig.'})')
print(f'Outliers détectés : {len(outliers)}')
print('Analyse terminée ✓')
```

RÉCAPITULATIF — Situation → Outil

Situation	Concept clé	Bibliothèque
Décrire le centre des données	Moyenne, Médiane, Mode	numpy
Mesurer l'étalement	Variance, Écart-type, IQR	numpy
Précision de la moyenne estimée	Erreur standard (SE)	scipy.stats
Vérifier la normalité	Shapiro-Wilk, QQ-plot, Skewness	scipy.stats
Comprendre le lien entre 2 variables	Covariance → Corrélacion	numpy / scipy
Comparer 1 groupe à une référence	Test t 1 éch. ou Wilcoxon	scipy.stats
Comparer 2 groupes indépendants	Test t ou Mann-Whitney U	scipy.stats
Comparer 3+ groupes	ANOVA (F) ou Kruskal-Wallis	scipy.stats
Lien entre 2 variables catégorielles	Chi ² (χ^2)	scipy.stats
Détecter des valeurs aberrantes	Z-score (normal) / IQR (asymétrique)	scipy / numpy
Estimer avec marge d'erreur	Intervalle de confiance 95%	scipy.stats
Charger et nettoyer les données	read_csv, dropna, fillna, astype	pandas

★ À retenir ★

Ce cours couvre l'ensemble des outils statistiques utilisés quotidiennement en **Data Science** et en **Analyse de données**.

Pour aller plus loin : régression, séries temporelles, ML supervisé.
Mais la maîtrise de ces bases est **indispensable avant tout**.